

DESIGN AND SIMULATION OF BASIC DIGITAL LOGIC CIRCUITS USING VERLOG/VHDL

BY :

NAME : BANDI THANISH KUMAR

COLLEGE : IIIT DHARWAD

PROJECT OBJECTIVE :

- Introduction to VLSI
- Digital Electronics Fundamentals
- CMOS Technology Basics
- Verilog/VHDL Basics

Tools Used:

- Verilog/VHDL
- Simulator : Vivado Simulator
- Waveform viewer
- Text Editor

UNDERSTANDING THE VLSI DESIGN FLOW :

What is VLSI?

VLSI (Very Large Scale Integration) is the technology of integrating millions to billions of transistors on a single integrated circuit (IC) to perform complex functions.

- Earlier : SSI, MSI, LSI
- Now : VLSI → ULSI
- Example : Microprocessors, memory chips, SoCs

In Simple Words:

VLSI = putting a complete electronic system on one chip

APPLICATIONS OF VLSI :

VLSI is used almost everywhere in modern electronics.

1.Consumer Electronics

1. Mobile phones
2. Smart TVs
3. Washing machines

2.Computing Systems

1. CPUs, GPUs
2. Laptops, servers

3.Communication Systems

1. Routers, modems
2. 4G/5G devices

4.Automotive Electronics

1. Engine control units (ECU)
2. Airbags, ABS systems

5.Medical Electronics

1. ECG, MRI machines
2. Wearable health devices

6.IoT & Embedded Systems

1. Smart sensors
2. Smart home devices

VLSI DESIGN FLOW (Specification → Design → Simulation) :

1. Specification

This is the first step in VLSI design.

- Defines what the chip should do
- Includes:
 - Functionality
 - Speed (clock frequency)
 - Power consumption
 - Area constraints
 - Technology node (e.g., 45 nm, 28 nm)

Output: **Design requirements document**

2. Architecture / Micro-architecture Design

- Break the system into **functional blocks**
 - ALU, registers, control unit, memory, etc.
- Decide:
 - Data path
 - Control logic
 - Pipelining (if any)

Output: **Block diagram**

3. RTL Design (Design using HDL) :

- Convert architecture into **HDL code**
- Languages used:
 - Verilog
 - VHDL
 - SystemVerilog
- Describes:
 - Registers
 - Combinational logic
 - Clocked behavior

Output: **RTL code**

4. Testbench Design

- Write a **testbench** to apply inputs to RTL
- No hardware → only for testing
- Generates:
 - Clock
 - Reset
 - Test vectors

Output: **Testbench code**

5. Functional Simulation :

- Simulate RTL using testbench

- Tools:

- ModelSim
- QuestaSim
- VCS

- Checks:

- Correct output
- Timing behavior at functional level
- Logic correctness

Output: **Waveforms & simulation results**

FLOWCHART SUMMARY :



Specification



Architecture Design



RTL Design (HDL Coding)



Testbench Design



Functional Simulation

ROLE OF HDL IN VLSI :

HDL (Hardware Description Language) is used to describe hardware behavior and structure.

Common HDLs:

- Verilog
- VHDL
- SystemVerilog

Role of HDL:

1.Design Description

1. Describe circuits at RTL level

2.Simulation & Verification

1. Verify functionality before fabrication

3.Synthesis Support

1. Convert HDL code into gate-level netlist

4.Technology Independence

1. Same HDL code can be synthesized for different technologies

5.Faster Design Cycle

1. Easy modification and debugging

6.Reusable Designs

1. Modules can be reused

***** Without HDL, designing complex VLSI chips is almost impossible*****

DIGITAL LOGIC DESIGN :

- **AND GATE :**
Description:

The AND gate produces HIGH output only when **both inputs are HIGH**.

A	B	$Y=A.B$
0	0	0
0	1	0
1	0	0
1	1	1

-
- **OR GATE :**
Description :

Output is HIGH when **at least one input is HIGH**

A	B	$Y=A+B$
0	0	0
0	1	1
1	0	1
1	1	1

- **NAND GATE :**

Description:

Output is the **NOT** of **AND** operation.

A	B	$Y = \overline{(A \cdot B)}$
0	0	1
0	1	1
1	0	1
1	1	0

- **NOR GATE :**

Description :

Output is the **NOT** of **OR** operation.

A	B	$Y = \overline{(A + B)}$
0	0	1
0	1	0
1	0	0
1	1	0

- **NOT GATE :**
Description:

Output is the **complement** of input.

A	$Y = \overline{A}$
0	1
1	0

- **XOR GATE :**
Description :

Output is HIGH when inputs are different.

A	B	$Y = A \oplus B$
0	0	1
0	1	0
1	0	0
1	1	0

Half Adder :

A Half Adder adds two single-bit binary numbers.

•**Sum (S)** = $A \oplus B = (A \text{ XOR } B) = (\bar{A}B + A\bar{B})$

•**Carry (C)** = $A \cdot B = (AB)$

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

CMOS LOGIC UNDERSTANDING

1. NMOS AND PMOS OPERATION :

NMOS Operation :

- NMOS uses **electrons** as charge carriers.
- It turns **ON** when:

$$V_{GS} > V_{TH}$$

- When ON, it provides a **low-resistance** path to ground (0).
- Used to **pull output LOW**.

States:

- **Cut-off:** $V_{GS} < V_{TH} \rightarrow \text{OFF}$
- **Triode:** small $V_{DS} \rightarrow$ behaves like a resistor
- **Saturation:** large $V_{DS} \rightarrow$ current source

PMOS Operation :

- PMOS uses **holes** as charge carriers.
- It turns **ON** when:

$$V_{SG} > |V_{TH}|$$

- When ON, it provides a **path to VDD (1)**.
- Used to **pull output HIGH**.

States:

- Cut-off:** OFF when gate is HIGH
- Triode & Saturation:** similar to NMOS but polarities reversed

KEY DIFFERENCE :

FEATURE	NMOS	PMOS
Carriers	Electrons	Holes
Turns ON	Gate High	Gate Low
Pulls	Output LOW	Output HIGH
Speed	Faster	Slower

2.CMOS Inverter Working :

A CMOS inverter consists of:

- PMOS at the top (connected to VDD)
- NMOS at the bottom (connected to GND)

Working:

Case 1: Input = 0 (LOW)

- PMOS → ON
- NMOS → OFF
- Output → 1 (HIGH)

Case 2: Input = 1 (HIGH)

- PMOS → OFF
- NMOS → ON
- Output → 0 (LOW)

Hence, output is the **inverse** of input.

TRUTH TABLE

INPUT	PMOS	NMOS	OUTPUT
0	ON	OFF	1
1	OFF	ON	0

3. Power Dissipation Concept in CMOS :

Power dissipation is the **energy** consumed by a circuit during operation.

Types of Power Dissipation:

(a) Static Power

- Occurs when circuit is **idle**
- Ideally **zero** in CMOS
- Caused by leakage currents

$$P_{static} = I_{leakage} \times V_{DD}$$

b) Dynamic Power

- Occurs during **switching**
- Due to charging and discharging of load capacitance

$$P_{dynamic} = \alpha C_L V_{DD}^2 f$$

Where:

- α = switching activity
- C_L =load capacitance
- f = frequency

(c) Short-Circuit Power

- Both NMOS and PMOS ON briefly during switching
- Causes direct path from VDD to GND

4. Why CMOS is Preferred in VLSI :

CMOS is the most widely used technology in VLSI because of the following advantages:

Advantages of CMOS:

1. Very Low Power Consumption

- 1. Almost zero static power

2. High Noise Immunity

- 1. Good logic level separation

3. High Packing Density

- 1. More transistors per chip

4. Scalability

- 1. Suitable for nanometer technologies

5. High Speed

- 1. Fast switching with low power

6. Wide Voltage Range

- 1. Works at low supply voltages

AND GATE

CODE :

testbench.sv

```
1 module tb;
2   reg A, B;
3   wire Y;
4
5   and_gate uut (.A(A), .B(B), .Y(Y));
6
7   initial begin
8     $dumpfile("dump.vcd");
9     $dumpvars(0, tb);
10
11     A = 0; B = 0;
12     #10 A = 0; B = 1;
13     #10 A = 1; B = 0;
14     #10 A = 1; B = 1;
15     #10 $finish;
16   end
17 endmodule
18
19
```

design.sv

```
1 module and_gate (
2   input A,
3   input B,
4   output Y
5 );
6   assign Y = A & B;
7 endmodule
8
9
```

(AND GATE) WAVEFORM RESULTS

EPWave

From: 0ns To: 40ns

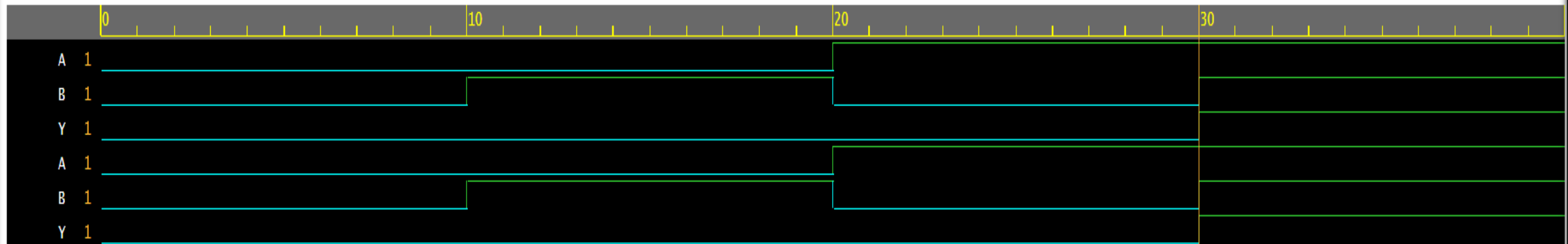
Get Signals Radix ▾ 🔍 🔍 100% ⏮ ⏭ ⚡ 20ns ⬆ ⬇ ⛔



EPWave

From: 0ns To: 40ns

Get Signals Radix ▾ 🔍 🔍 100% ⏮ ⏭ ⚡ 30ns ⬆ ⬇ ⛔



OR GATE

CODE :

testbench.sv

```
1 module tb;
2   reg A, B;
3   wire Y;
4
5   or_gate uut (.A(A), .B(B), .Y(Y));
6
7   initial begin
8     $dumpfile("dump.vcd");
9     $dumpvars(0, tb);
10
11     A = 0; B = 0;
12     #10 A = 0; B = 1;
13     #10 A = 1; B = 0;
14     #10 A = 1; B = 1;
15     #10 $finish;
16   end
17 endmodule
18
```

design.sv

```
1 module or_gate (
2   input A,
3   input B,
4   output Y
5 );
6   assign Y = A | B;
7 endmodule
8
```

(OR GATE) WAVEFORM RESULTS

EPWave

From: 0ns To: 40ns

Get Signals

Radix ▾



100%



20ns



EPWave

From: 0ns To: 40ns

Get Signals

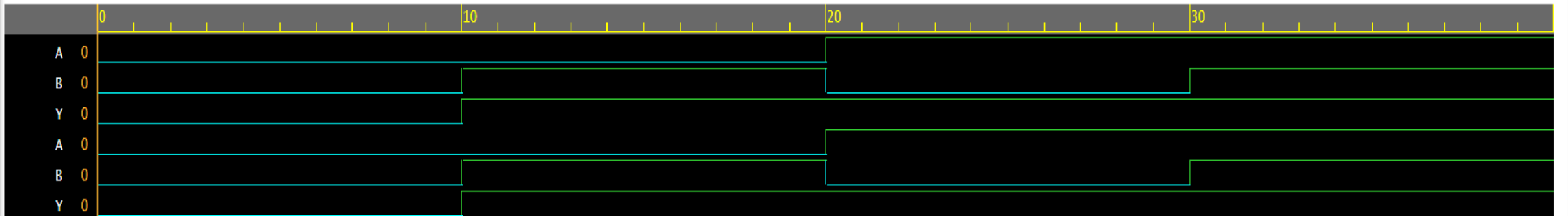
Radix ▾



100%



0ns



NOT GATE

CODE :

testbench.sv

```
1 module tb;
2   reg A;
3   wire Y;
4
5   not_gate uut (.A(A), .Y(Y));
6
7   initial begin
8     $dumpfile("dump.vcd");
9     $dumpvars(0, tb);
10
11     A = 0;
12     #10 A = 1;
13     #10 $finish;
14   end
15 endmodule
16
17
```

design.sv

```
1 module not_gate (
2   input A,
3   output Y
4 );
5   assign Y = ~A;
6 endmodule
7
8
```

(NOT GATE)WAVEFORM RESULTS

EPWave

From: 0ns To: 20ns

Get Signals

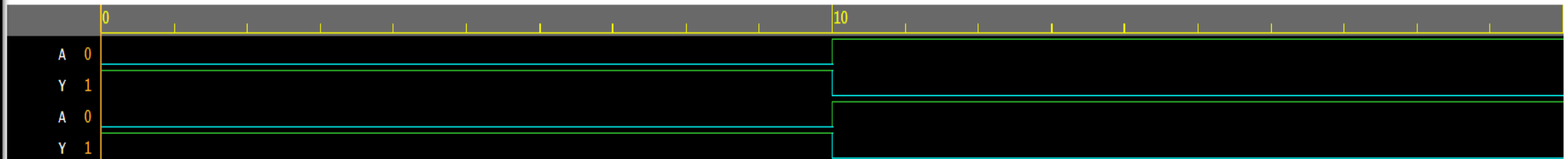
Radix ▾



100%



⚡ 0ns



HALF ADDER

CODE :

testbench.sv



```
1 // Testbench for Half Adder
2
3 module half_adder_tb;
4
5     reg A, B;
6     wire Sum, Carry;
7
8     // Instantiate the Half Adder
9     half_adder uut (
10         .A(A),
11         .B(B),
12         .Sum(Sum),
13         .Carry(Carry)
14     );
15
16     initial begin
17         // VCD file for waveform
18         $dumpfile("half_adder.vcd");
19         $dumpvars(0, half_adder_tb);
20
21         // Test cases
22         A = 0; B = 0; #10;
23         A = 0; B = 1; #10;
24         A = 1; B = 0; #10;
25         A = 1; B = 1; #10;
26
27         $finish;
28     end
29
30 endmodule
31
```

design.sv



```
1 // Half Adder Design
2 // Inputs : A, B
3 // Outputs: Sum, Carry
4
5 module half_adder (
6     input A,
7     input B,
8     output Sum,
9     output Carry
10 );
11
12     assign Sum = A ^ B; // XOR operation
13     assign Carry = A & B; // AND operation
14
15 endmodule
16
17 |
```

(HALF ADDER)WAVEFORM RESULTS

EPWave

From: 0ns To: 40ns

Get Signals

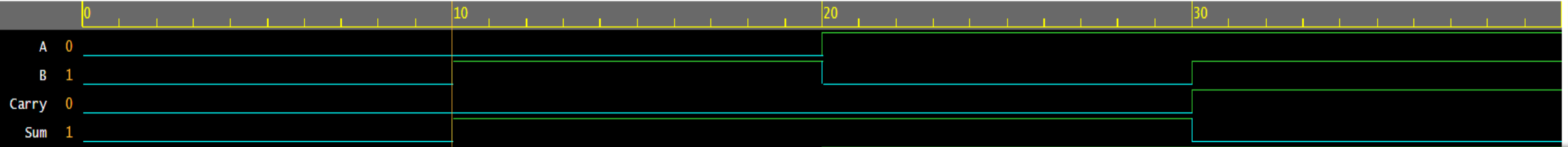
Radix ▾



100%



⚡ 10ns



EPWave

From: 0ns To: 40ns

Get Signals

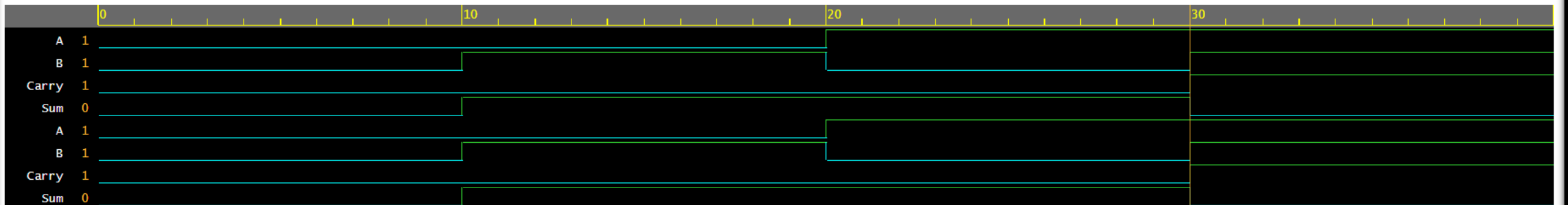
Radix ▾



100%



⚡ 30ns



The background features abstract, overlapping green geometric shapes in various shades of green, creating a modern, layered effect on the right side of the slide.

VERIFICATION

VERIFICATION

VERIFICATION (FOR ALL LOGIC GATES) :

Verification Procedure :

- 1.The logic gate is designed using Verilog HDL.
- 2.A testbench is written to apply **all possible input combinations**.
- 3.The simulation is run using a Verilog simulator.
- 4.The output is observed in the **console and waveform**.
- 5.The obtained output is compared with the **truth table**.
- 6.Since the output matches the truth table for all inputs, the gate is **verified**.

Observation

- Inputs are changed one by one.
- Output changes correctly according to the truth table.
- No incorrect or unknown output is observed.

Result :

The given logic gate is verified successfully.

THANK YOU